Mocking mit Mockito

Carsten Gips (HSBI)

TL;DR
Häufig hat man es in Softwaretests mit dem Problem zu tun, dass die zu testenden Klassen von anderen, noch nicht implementierten Klassen oder von zufälligen oder langsamen Operationen abhängen.
In solchen Situationen kann man auf "Platzhalter" für diese Abhängigkeiten zurückgreifen. Dies können einfache Stubs sein, also Objekte, die einfach einen festen Wert bei einem Methodenaufruf zurückliefern oder Mocks, wo man auf die Argumente eines Methodenaufrufs reagieren kann und passende unterschiedliche Rückgabewerte zurückgeben kann.
Mockito ist eine Java-Bibliothek, die zusammen mit JUnit das Mocking von Klassen in Java erlaubt. Man kann hier zusätzlich auch die Interaktion mit dem gemockten Objekt überprüfen und testen, ob eine bestimmte Methode mit bestimmten Argumenten aufgerufen wurde und wie oft.

Videos

- VL Mocking
- Demo Mocking: Stubs
- Demo Mocking: Mocks
- Demo Mocking: Spy
- Demo Mocking: verify()
- Demo Mocking: Matcher

Unterlagen

- PDF-Version

---

1 Motivation: Entwicklung einer Studi-/Prüfungsverwaltung

1.1 Szenario

Zwei Teams entwickeln eine neue Studi-/Prüfungsverwaltung für die Hochschule. Ein Team modelliert dabei die Studierenden, ein anderes Team modelliert die Prüfungsverwaltung LSF.

- Team A:

```java
public class Studi {
    String name;  LSF lsf;

    public Studi(String name, LSF lsf) {
        this.name = name;  this.lsf = lsf;
    }

    public boolean anmelden(String modul) { return lsf.anmelden(name, modul); }
    public boolean einsicht(String modul) { return lsf.ergebnis(name, modul) > 50; }
}
```

- Team B:

```java
public class LSF {
    public boolean anmelden(String name, String modul) { throw new
        UnsupportedOperationException(); }
    public int ergebnis(String name, String modul) { throw new
        UnsupportedOperationException(); }
}
```

Team B kommt nicht so recht vorwärts, Team A ist fertig und will schon testen.

Wie kann Team A seinen Code testen?

Optionen:

- Gar nicht testen?!
- Das LSF selbst implementieren? Wer pflegt das dann? => manuell implementierte Stubs
- Das LSF durch einen Mock ersetzen => Einsatz der Bibliothek "mockito"

1.2 Motivation Mocking und Mockito

Mockito ist ein Mocking-Framework für JUnit. Es simuliert das Verhalten eines realen Objektes oder einer realen Methode.

Wofür brauchen wir denn jetzt so ein Mocking-Framework überhaupt?

Wir wollen die Funktionalität einer Klasse isoliert vom Rest testen können. Dabei stören uns aber bisher so ein paar Dinge:

- Arbeiten mit den echten Objekten ist langsam (zum Beispiel aufgrund von Datenbankenzugriffen)
- Objekte beinhalten oft komplexe Abhängigkeiten, die in Tests schwer abzudecken sind
- Manchmal existiert der zu testende Teil einer Applikation auch noch gar nicht, sondern es gibt nur die Interfaces.
- Oder es gibt unschöne Seiteneffekte beim Arbeiten mit den realen Objekten. Zum Beispiel könnte es sein, das immer eine E-Mail versendet wird, wenn wir mit einem Objekt interagieren.

In solchen Situationen wollen wir eine Möglichkeit haben, das Verhalten eines realen Objektes bzw. der Methoden zu simulieren, ohne dabei die originalen Methoden aufrufen zu müssen. (Manchmal möchte man das dennoch, aber dazu später mehr…)

Und genau hier kommt Mockito ins Spiel. Mockito hilft uns dabei, uns von den externen Abhängigkeiten zu lösen, indem es sogenannte Mocks, Stubs oder Spies anbietet, mit denen sich das Verhalten der realen Objekte simulieren/überwachen und testen lässt.

1.3 Aber was genau ist denn jetzt eigentlich Mocking?

Ein Mock-Objekt ("etwas vortäuschen") ist im Software-Test ein Objekt, das als Platzhalter (Attrappe) für das echte Objekt verwendet wird.

Mocks sind in JUnit-Tests immer dann nützlich, wenn man externe Abhängigkeiten hat, auf die der eigene Code zugreift. Das können zum Beispiel externe APIs sein oder Datenbanken etc. … Mocks helfen einem beim Testen nun dabei, sich von diesen externen Abhängigkeiten zu lösen und seine Softwarefunktionalität dennoch schnell und effizient testen zu können ohne evtl. auftretende Verbindungsfehler oder andere mögliche Seiteneffekte der externen Abhängigkeiten auszulösen.

Dabei simulieren Mocks die Funktionalität der externen APIs oder Datenbankzugriffe. Auf diese Weise ist es möglich Softwaretests zu schreiben, die scheinbar die gleichen Methoden aufrufen, die sie auch im regulären Softwarebetrieb nutzen würden, allerdings werden diese wie oben erwähnt allerdings für die Tests nur simuliert.

Mocking ist also eine Technik, die in Softwaretests verwendet wird, in denen die gemockten Objekte anstatt der realen Objekte zu Testzwecken genutzt werden. Die gemockten Objekte liefern dabei bei einem vom Programmierer bestimmten (Dummy-) Input, einen dazu passenden gelieferten (Dummy-) Output, der durch seine vorhersagbare Funktionalität dann in den eigentlichen Testobjekten gut für den Test nutzbar ist.

Dabei ist es von Vorteil die drei Grundbegriffe “Mock”, “Stub” oder “Spy”, auf die wir in der Vorlesung noch häufiger treffen werden, voneinander abgrenzen und unterscheiden zu können.

1.4 Dabei bezeichnet ein

- **Stub**: Ein Stub ist ein Objekt, dessen Methoden nur mit einer minimalen Logik für den Test implementiert wurden. Häufig werden dabei einfach feste (konstante) Werte zurückgeliefert, d.h. beim Aufruf einer Methode wird unabhängig von der konkreten Eingabe immer die selbe Ausgabe zurückgeliefert.
- **Mock**: Ein Mock ist ein Objekt, welches im Gegensatz zum Stub bei vorher definierten Funktionsaufrufen mit vorher definierten Argumente eine definierte Rückgabe liefert.
- **Spy**: Ein Spy ist ein Objekt, welches Aufrufe und übergebene Werte protokolliert und abfragbar macht. Es ist also eine Art Wrapper um einen Stub oder einen Mock.

1.5 Mockito Setup

- Gradle: `build.gradle`

```
dependencies {
    implementation 'junit:junit:4.13.2'
    implementation 'org.mockito:mockito-core:4.5.1'
}
```

- Maven: `pom.xml`

```
<dependencies>
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
    </dependency>
        <groupId>org.mockito</groupId>
        <artifactId>mockito-core</artifactId>
```

```
        <version>4.5.1</version>
    </dependency>
</dependencies>
```

2 Manuell Stubs implementieren

Team A könnte manuell das LSF rudimentär implementieren (nur für die Tests, einfach mit festen Rückgabewerten): **Stubs**

```
public class StudiStubTest {
    Studi studi;  LSF lsf;

    @Before
    public void setUp() { lsf = new LsfStub();  studi = new Studi("Harald", lsf
        ); }

    @Test
    public void testAnmelden() { assertTrue(studi.anmelden("PM-Dungeon")); }
    @Test
    public void testEinsicht() { assertTrue(studi.einsicht("PM-Dungeon")); }


    // Stub für das noch nicht fertige LSF
    class LsfStub extends LSF {
        public boolean anmelden(String name, String modul) { return true; }
        public int ergebnis(String name, String modul) { return 80; }
    }
}
```

Problem: Wartung der Tests (wenn das richtige LSF fertig ist) und Wartung der Stubs (wenn sich die Schnittstelle des LSF ändert, muss auch der Stub nachgezogen werden).

Problem: Der Stub hat nur eine Art minimale Default-Logik (sonst könnte man ja das LSF gleich selbst implementieren). Wenn man im Test andere Antworten braucht, müsste man einen weiteren Stub anlegen …

Demo hsbi.StudiStubTest

3 Mockito: Mocking von ganzen Klassen

Lösung: Mocking der Klasse `LSF` mit Mockito für den Test von `Studi`: `mock()`.

```
public class StudiMockTest {
    Studi studi;  LSF lsf;

    @Before
    public void setUp() { lsf = mock(LSF.class);  studi = new Studi("Harald",
        lsf); }

    @Test
    public void testAnmelden() {
```

```
9           when(lsf.anmelden(anyString(), anyString())).thenReturn(true);
10          assertTrue(studi.anmelden("PM-Dungeon"));
11      }
12
13      @Test
14      public void testEinsichtI() {
15          when(lsf.ergebnis("Harald", "PM-Dungeon")).thenReturn(80);
16          assertTrue(studi.einsicht("PM-Dungeon"));
17      }
18      @Test
19      public void testEinsichtII() {
20          when(lsf.ergebnis("Harald", "PM-Dungeon")).thenReturn(40);
21          assertFalse(studi.einsicht("PM-Dungeon"));
22      }
23  }
```

Der Aufruf `mock(LSF.class)` erzeugt einen Mock der Klasse (oder des Interfaces) `LSF`. Dabei wird ein Objekt vom Typ `LSF` erzeugt, mit dem man dann wie mit einem normalen Objekt weiter arbeiten kann. Die Methoden sind allerdings nicht implementiert …

Mit Hilfe von `when().thenReturn()` kann man definieren, was genau beim Aufruf einer bestimmten Methode auf dem Mock passieren soll, d.h. welcher Rückgabewert entsprechend zurückgegeben werden soll. Hier kann man dann für bestimmte Argumentwerte andere Rückgabewerte definieren. `when(lsf.ergebnis("Harald", "PM-Dungeon")).thenReturn(80)` gibt also für den Aufruf von `ergebnis` mit den Argumenten `"Harald"` und `"PM-Dungeon"` auf dem Mock `lsf` den Wert 80 zurück.

Dies kann man in weiten Grenzen flexibel anpassen.

Mit Hilfe der Argument-Matcher `anyString()` wird jedes String-Argument akzeptiert.

Demo hsbi.StudiMockTest

4 Mockito: Spy = Wrapper um ein Objekt

Team B hat das `LSF` nun implementiert und Team A kann es endlich für die Tests benutzen. Aber das `LSF` hat eine Zufallskomponente (`ergebnis()`). Wie kann man nun die Reaktion des Studis testen (`einsicht()`)?

Lösung: Mockito-Spy als partieller Mock einer Klasse (Wrapper um ein Objekt): `spy()`.

```
1   public class StudiSpyTest {
2       Studi studi;  LSF lsf;
3
4       @Before
5       public void setUp() { lsf = spy(LSF.class);  studi = new Studi("Harald",
            lsf); }
6
7       @Test
8       public void testAnmelden() { assertTrue(studi.anmelden("PM-Dungeon")); }
9
10      @Test
11      public void testEinsichtI() {
12          doReturn(80).when(lsf).ergebnis("Harald", "PM-Dungeon");
```

```
13          assertTrue(studi.einsicht("PM-Dungeon"));
14      }
15      @Test
16      public void testEinsichtII() {
17          doReturn(40).when(lsf).ergebnis("Harald", "PM-Dungeon");
18          assertFalse(studi.einsicht("PM-Dungeon"));
19      }
20  }
```

Der Aufruf `spy(LSF.class)` erzeugt einen Spy um ein Objekt der Klasse `LSF`. Dabei bleiben zunächst die Methoden in `LSF` erhalten und können aufgerufen werden, sie können aber auch mit einem (partiellen) Mock überlagert werden. Der Spy zeichnet wie der Mock die Interaktion mit dem Objekt auf.

Mit Hilfe von `doReturn().when()` kann man definieren, was genau beim Aufruf einer bestimmten Methode auf dem Spy passieren soll, d.h. welcher Rückgabewert entsprechend zurückgegeben werden soll. Hier kann man analog zum Mock für bestimmte Argumentwerte andere Rückgabewerte definieren. `doReturn(40).when(lsf).ergebnis("Harald", "PM-Dungeon")` gibt also für den Aufruf von `ergebnis` mit den Argumenten `"Harald"` und `"PM-Dungeon"` auf dem Spy `lsf` den Wert 40 zurück.

Wenn man die Methoden nicht mit einem partiellen Mock überschreibt, dann wird einfach die originale Methode aufgerufen (Beispiel: In `studi.anmelden("PM-Dungeon")` wird `lsf.anmelden("Harald", "PM-Dungeon")` aufgerufen.).

Auch hier können Argument-Matcher wie `anyString()` eingesetzt werden.

Demo hsbi.StudiSpyTest

5 Wurde eine Methode aufgerufen?

```
1   public class VerifyTest {
2       @Test
3       public void testAnmelden() {
4           LSF lsf = mock(LSF.class);  Studi studi = new Studi("Harald", lsf);
5
6           when(lsf.anmelden("Harald", "PM-Dungeon")).thenReturn(true);
7
8           assertTrue(studi.anmelden("PM-Dungeon"));
9
10
11          verify(lsf).anmelden("Harald", "PM-Dungeon");
12          verify(lsf, times(1)).anmelden("Harald", "PM-Dungeon");
13
14          verify(lsf, atLeast(1)).anmelden("Harald", "PM-Dungeon");
15          verify(lsf, atMost(1)).anmelden("Harald", "PM-Dungeon");
16
17          verify(lsf, never()).ergebnis("Harald", "PM-Dungeon");
18
19          verifyNoMoreInteractions(lsf);
20      }
21  }
```

Mit der Methode `verify()` kann auf einem Mock oder Spy überprüft werden, ob und wie oft und in welcher Reihenfolge Methoden aufgerufen wurden und mit welchen Argumenten. Auch hier lassen sich wieder Argument-Matcher wie `anyString()` einsetzen.

Ein einfaches `verify(mock)` prüft dabei, ob die entsprechende Methode exakt einmal vorher aufgerufen wurde. Dies ist äquivalent zu `verify(mock, times(1))`. Analog kann man mit den Parametern `atLeast()` oder `atMost` bestimmte Unter- oder Obergrenzen für die Aufrufe angeben und mit `never()` prüfen, ob es gar keinen Aufruf vorher gab.

`verifyNoMoreInteractions(lsf)` ist interessant: Es ist genau dann **`true`**, wenn es außer den vorher abgefragten Interaktionen keinerlei sonstigen Interaktionen mit dem Mock oder Spy gab.

```
LSF lsf = mock(LSF.class);
Studi studi = new Studi("Harald", lsf);

when(lsf.anmelden("Harald", "PM-Dungeon")).thenReturn(true);

InOrder inOrder = inOrder(lsf);

assertTrue(studi.anmelden("PM-Dungeon"));
studi.anmelden("Wuppie");

inOrder.verify(lsf).anmelden("Harald", "Wuppie");
inOrder.verify(lsf).anmelden("Harald", "PM-Dungeon");
```

Mit `InOrder` lassen sich Aufrufe auf einem Mock/Spy oder auch auf verschiedenen Mocks/Spies in eine zeitliche Reihenfolge bringen und so überprüfen.

Demo hsbi.VerifyTest

6 Fangen von Argumenten

```
public class MatcherTest {
    @Test
    public void testAnmelden() {
        LSF lsf = mock(LSF.class);  Studi studi = new Studi("Harald", lsf);

        when(lsf.anmelden(anyString(), anyString())).thenReturn(false);
        when(lsf.anmelden("Harald", "PM-Dungeon")).thenReturn(true);

        assertTrue(studi.anmelden("PM-Dungeon"));
        assertFalse(studi.anmelden("Wuppie?"));

        verify(lsf, times(1)).anmelden("Harald", "PM-Dungeon");
        verify(lsf, times(1)).anmelden("Harald", "Wuppie?");

        verify(lsf, times(2)).anmelden(anyString(), anyString());
        verify(lsf, times(1)).anmelden(eq("Harald"), eq("Wuppie?"));
        verify(lsf, times(2)).anmelden(argThat(new MyHaraldMatcher()),
            anyString());
    }

```

```
20
21      class MyHaraldMatcher implements ArgumentMatcher<String> {
22          public boolean matches(String s) { return s.equals("Harald"); }
23      }
24  }
```

Sie können die konkreten Argumente angeben, für die der Aufruf gelten soll. Alternativ können Sie mit vordefinierten `ArgumentMatchers` wie `anyString()` beispielsweise auf beliebige Strings reagieren oder selbst einen eigenen `ArgumentMatcher<T>` für Ihren Typ `T` erstellen und nutzen.

Wichtig: Wenn Sie für einen Parameter einen `ArgumentMatcher` einsetzen, müssen Sie für die restlichen Parameter der Methode dies ebenfalls tun. Sie können keine konkreten Argumente mit `ArgumentMatcher` mischen.

Sie finden viele weitere vordefinierte Matcher in der Klasse `ArgumentMatchers`. Mit der Klasse `ArgumentCaptor<T>` finden Sie eine alternative Möglichkeit, auf Argumente in gemockten Methoden zu reagieren. Schauen Sie sich dazu die Javadoc von Mockito an.

Demo hsbi.MatcherTest

7 Ausblick: PowerMock

Mockito sehr mächtig, aber unterstützt (u.a.) keine

- Konstruktoren
- private Methoden
- final Methoden
- static Methoden (ab Version 3.4.0 scheint auch Mockito statische Methoden zu unterstützen)

=> Lösung: PowerMock

8 Ausführlicheres Beispiel: WuppiWarenlager

Credits: Der Dank für die Erstellung des nachfolgenden Beispiels und Textes geht an @jedi101.

Demo: WuppiWarenlager (wuppie.stub)

Bei dem gezeigten Beispiel unseres `WuppiStores` sieht man, dass dieser normalerweise von einem fertigen Warenlager die Wuppis beziehen möchte. Da dieses Lager aber noch nicht existiert, haben wir uns kurzerhand einfach einen Stub von unserem `IWuppiWarenlager`-Interface erstellt, in dem wir zu Testzwecken händisch ein Paar Wuppis ins Lager geräumt haben.

Das funktioniert in diesem Mini-Testbeispiel ganz gut aber, wenn unsere Stores erst einmal so richtig Fahrt aufnehmen und wir irgendwann weltweit Wuppis verkaufen, wird der Code des `IWuppiWarenlagers` wahrscheinlich sehr schnell viel komplexer werden, was unweigerlich dann zu Maintenance-Problemen unserer

händisch angelegten Tests führt. Wenn wir zum Beispiel einmal eine Methode hinzufügen wollen, die es uns ermöglicht, nicht immer alle Wuppis aus dem Lager zu ordern oder vielleicht noch andere Methoden, die Fluppis orderbar machen, hinzufügen, müssen wir immer dafür sorgen, dass wir die getätigten Änderungen händisch in den Stub des Warenlagers einpflegen.

Das will eigentlich niemand…

8.1 Einsatz von Mockito

Aber es gibt da einen Ausweg. Wenn es komplexer wird, verwenden wir Mocks.

Bislang haben wir noch keinen Gebrauch von Mockito gemacht. Das ändern wir nun.

Demo: WuppiWarenlager (wuppie.mock)

Wie in diesem Beispiel gezeigt, müssen wir nun keinen Stub mehr von Hand erstellen, sondern überlassen dies Mockito.

```
1  IWuppiWarenlager lager = mock(IWuppiWarenlager.class);
```

Anschließend können wir, ohne die Methode `getAllWuppis()` implementiert zu haben, dennoch so tun als, ob die Methode eine Funktionalität hätte.

```
1  // Erstellen eines imaginären Lagerbestands.
2  List<String> wuppisImLager = Arrays.asList("GruenerWuppi","RoterWuppi");
3  when(lager.getAlleWuppis()).thenReturn(wuppisImLager);
```

Wann immer nun die Methode `getAlleWuppis()` des gemockten Lagers aufgerufen wird, wird dieser Aufruf von Mockito abgefangen und wie oben definiert verändert. Das Ergebnis können wir abschließend einfach in unserem Test testen:

```
1  // Erzeugen des WuppiStores.
2  WuppiStore wuppiStore = new WuppiStore(lager);
3
4  // Bestelle alle Wuppis aus dem gemockten Lager List<String>
5  bestellteWuppis = wuppiStore.bestelleAlleWuppis(lager);
6
7  // Hat die Bestellung geklappt?
8  assertEquals(2,bestellteWuppis.size());
```

8.2 Mockito Spies

Manchmal möchten wir allerdings nicht immer gleich ein ganzes Objekt mocken, aber dennoch Einfluss auf die aufgerufenen Methoden eines Objekts haben, um diese testen zu können. Vielleicht gibt es dabei ja sogar eine Möglichkeit unsere JUnit-Tests, mit denen wir normalerweise nur Rückgabewerte von Methoden testen können, zusätzlich auch das Verhalten also die Interaktionen mit einem Objekt beobachtbar zu machen. Somit wären diese Interaktionen auch testbar.

Und genau dafür bietet Mockito eine Funktion: der sogenannte “Spy”.

Dieser Spion erlaubt es uns nun zusätzlich das Verhalten zu testen. Das geht in die Richtung von BDD - Behavior Driven Development.

Demo: WuppiWarenlager (wuppie.spy)

```
// Spion erstellen, der unser wuppiWarenlager überwacht.
this.wuppiWarenlager = spy(WuppiWarenlager.class);
```

Hier hatten wir uns einen Spion erzeugt, mit dem sich anschließend das Verhalten verändern lässt:

```
when(wuppiWarenlager.getAlleWuppis()).thenReturn(Arrays.asList(new Wuppi("
    Wuppi007")));
```

Aber auch der Zugriff lässt sich kontrollieren/testen:

```
verify(wuppiWarenlager).addWuppi(normalerWuppi);
verifyNoMoreInteractions(wuppiWarenlager);
```

Die normalen Testmöglichkeiten von JUnit runden unseren Test zudem ab.

```
assertEquals(1,wuppiWarenlager.lager.size());
```

9 Mockito und Annotationen

In Mockito können Sie wie oben gezeigt mit `mock()` und `spy()` neue Mocks bzw. Spies erzeugen und mit `verify()` die Interaktion überprüfen und mit `ArgumentMatcher<T>` bzw. den vordefinierten `ArgumentMatchers` auf Argumente zuzugreifen bzw. darauf zu reagieren.

Zusätzlich/alternativ gibt es in Mockito zahlreiche Annotationen, die **ersatzweise** statt der genannten Methoden genutzt werden können. Hier ein kleiner Überblick über die wichtigsten in Mockito verwendeten Annotation:

- `@Mock` wird zum Markieren des zu mockenden Objekts verwendet.

  ```
  @Mock
  WuppiWarenlager lager;
  ```

- `@RunWith(MockitoJUnitRunner.class)` ist der entsprechende JUnit-Runner, wenn Sie Mocks mit `@Mock` anlegen.

  ```
  @RunWith(MockitoJUnitRunner.class)
  public class ToDoBusinessMock {...}
  ```

- `@Spy` erlaubt das Erstellen von partiell gemockten Objekten. Dabei wird eine Art Wrapper um das zu mockende Objekt gewickelt, der dafür sorgt, dass alle Methodenaufrufe des Objekts an den Spy delegiert werden. Diese können über den Spion dann abgefangen/verändert oder ausgewertet werden.

  ```
  @Spy
  ArrayList<Wuppi> arrayListenSpion;
  ```

- `@InjectMocks` erlaubt es, Parameter zu markieren, in denen Mocks und/oder Spies injiziert werden. Mockito versucht dann (in dieser Reihenfolge) per Konstruktorinjektion, Setterinjektion oder Propertyinjektion die Mocks zu injizieren. Weitere Informationen darüber findet man hier: Mockito Dokumentation

  Anmerkung: Es ist aber nicht ratsam "Field- oder Setterinjection" zu nutzen, da man nur bei der Verwendung von "Constructorinjection" sicherstellen kann, das eine Klasse nicht ohne die eigentlich notwendigen Parameter instanziiert wurde.

  ```
  1 @InjectMocks
  2 Wuppi fluppi;
  ```

- `@Captor` erlaubt es, die Argumente einer Methode abzufangen/auszuwerten. Im Zusammenspiel mit Mockitos `verify()`-Methode kann man somit auch die einer Methode übergebenen Argumente verifizieren.

  ```
  1 @Captor
  2 ArgumentCaptor<String> argumentCaptor;
  ```

- `@ExtendWith(MockitoExtension.class)` wird in JUnit5 verwendet, um die Initialisierung von Mocks zu vereinfachen. Damit entfällt zum Beispiel die noch unter JUnit4 nötige Initialisierung der Mocks durch einen Aufruf der Methode `MockitoAnnotations.openMocks()` im Setup des Tests (`@Before` bzw. `@BeforeEach`).

9.1 Prüfen der Interaktion mit *verify()*

Mit Hilfe der umfangreichen `verify()`-Methoden, die uns Mockito mitliefert, können wir unseren Code unter anderem auf unerwünschte Seiteneffekte testen. So ist es mit `verify` zum Beispiel möglich abzufragen, ob mit einem gemockten Objekt interagiert wurde, wie damit interagiert wurde, welche Argumente dabei übergeben worden sind und in welcher Reihenfolge die Interaktionen damit erfolgt sind.

Hier nur eine kurze Übersicht über das Testen des Codes mit Hilfe von Mockitos `verify()`-Methoden.

```
1 @Test
2 public void testVerify_DasKeineInteraktionMitDerListeStattgefundenHat() {
3     // Testet, ob die spezifizierte Interaktion mit der Liste nie stattgefunden
          hat.
4     verify(fluppisListe, never()).clear();
5 }
```

```
1 @Test
2 public void testVerify_ReihenfolgeDerInteraktionenMitDerFluppisListe() {
3     // Testet, ob die Reihenfolge der spezifizierten Interaktionen mit der
          Liste eingehalten wurde.
4     fluppisListe.clear();
5     InOrder reihenfolge = inOrder(fluppisListe);
6     reihenfolge.verify(fluppisListe).add("Fluppi001");
7     reihenfolge.verify(fluppisListe).clear();
8 }
```

```
1 @Test
2 public void testVerify_FlexibleArgumenteBeimZugriffAufFluppisListe() {
```

```
3      // Testet, ob schon jemals etwas zu der Liste hinzugefügt wurde.
4      // Dabei ist es egal welcher String eingegeben wurde.
5      verify(fluppisListe).add(anyString());
6  }
```

```
1  @Test
2  public void testVerify_InteraktionenMitHilfeDesArgumentCaptor() {
3      // Testet, welches Argument beim Methodenaufruf übergeben wurde.
4      fluppisListe.addAll(Arrays.asList("BobDerBaumeister"));
5      ArgumentCaptor<List> argumentMagnet = ArgumentCaptor.forClass(FluppisListe.
           class);
6      verify(fluppisListe).addAll(argumentMagnet.capture());
7      List<String> argumente = argumentMagnet.getValue();
8      assertEquals("BobDerBaumeister", argumente.get(0));
9  }
```

Demo: WuppiWarenlager (wuppie.verify)

10 Wrap-Up

- Gründliches Testen ist ebenso viel Aufwand wie Coden!

- Mockito ergänzt JUnit:
  - Mocken ganzer Klassen (`mock()`, `when().thenReturn()`)
  - Wrappen von Objekten (`spy()`, `doReturn().when()`)
  - Auswerten, wie häufig Methoden aufgerufen wurden (`verify()`)
  - Auswerten, mit welchen Argumenten Methoden aufgerufen wurden (`anyString`)

Lernziele

- k2: Begriffe: Mocking, Mock, Stub, Spy
- k3: Erzeugen eines Mocks in Mockito
- k3: Erzeugen eines Spies in Mockito
- k3: Prüfen von Interaktion mit verify()
- k3: Einsatz von ArgumentMatcher

Challenges

Betrachten Sie die drei Klassen `Utility.java`, `Evil.java` und `UtilityTest.java`:

```java
public class Utility {
    private int intResult = 0;
    private Evil evilClass;

    public Utility(Evil evilClass) {
        this.evilClass = evilClass;
    }

    public void evilMethod() {
        int i = 2 / 0;
    }

    public int nonEvilAdd(int a, int b) {
        return a + b;
    }

    public int evilAdd(int a, int b) {
        evilClass.evilMethod();
        return a + b;
    }

    public void veryEvilAdd(int a, int b) {
        evilMethod();
        evilClass.evilMethod();
        intResult = a + b;
    }

    public int getIntResult() {
        return intResult;
    }
}

public class Evil {
    public void evilMethod() {
        int i = 3 / 0;
    }
}

public class UtilityTest {
    private Utility utilityClass;
    // Initialisieren Sie die Attribute entsprechend vor jedem Test.

    @Test
    void test_nonEvilAdd() {
        Assertions.assertEquals(10, utilityClass.nonEvilAdd(9, 1));
    }

    @Test
    void test_evilAdd() {
        Assertions.assertEquals(10, utilityClass.evilAdd(9, 1));
    }

    @Test
    void test_veryEvilAdd() {
        utilityClass.veryEvilAdd(9, 1);
        Assertions.assertEquals(10, utilityClass.getIntResult());
    }
}
```

Testen Sie die Methoden `nonEvilAdd`, `evilAdd` und `veryEvilAdd` der Klasse `Utility.java` mit dem JUnit- und dem Mockito-Framework.

Vervollständigen Sie dazu die Klasse `UtilityTest.java` und nutzen Sie Mocking mit Mockito, um die Tests zum Laufen zu bringen. Die Tests dürfen Sie entsprechend verändern, aber die Aufrufe aus der Vorgabe müssen erhalten

---

Quellen

S. Faber and B. Dutheil and R. Winterhalter and T.v.d. Lippe. 2022. „Mockito“. 2022. https://github.com/mockito/mockito.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 09509bb (tooling: shift headings (use h1 as top-level headings) (#993), 2025-04-23)